

books-40c — The Watchman: Never Missing a Garnishment Deadline

For: Michael, Greenway Marijuana **Slice:** books-40c (finishing the garnishments feature, as you asked) **Screens:** Books → Garnishments, plus your email and your phone **Status:** built, wired, reachable, and proved by twenty-four deliberate acts of sabotage that the test suite caught every single time

The one-paragraph version

You asked how you would be notified about ending garnishments and about filing the twenty-day answer. The honest answer was that you would not have been. The system knew both deadlines — it had known them since books-38 — and it told you about them exactly once, on the screen where you typed the order in, for about thirty seconds, and then never again for the rest of that order's life. This change makes those deadlines follow you. They now appear on the garnishments board every time you open it, and they arrive by email and phone notification on a schedule that gets louder as the deadline gets closer. Critically, it also gives you a way to make them **stop** — by recording that you actually filed the answer. There is deliberately no "dismiss" button, and the reason why is the most important thing in this document.

First, the honest part: what was wrong

When I went looking for where to add the reminders, I found something I did not expect. Two functions already existed and were already correct:

- `answerDeadlineFor()` — works out your twenty-day answer deadline
- `expiryOutlookFor()` — works out when a creditor writ dies at sixty days

Both were written months ago. Both were tested. Both were right. And both were imported by exactly **one** file in the entire application: the form you use to type in a new order.

Sit with what that means, because it is worse than it first sounds. The twenty- day clock was calculated the moment you entered the order, displayed on screen while you were typing, and then thrown away. Day two, day ten, day nineteen, day forty — nothing anywhere in the system ever looked at that date again. Not the board. Not an email. Nothing. The software knew the exact date you were exposed to a hundred-percent liability, and it kept that information to itself.

This is the third time in this run of work I have found the same shape of defect: correct code that nothing calls. In books-38 the reader never selected `served_date`. In books-40b three lifecycle buttons existed and were wired to nothing. Every test passed each time, because no test asked the only question that mattered — *can this actually reach Michael?* There are now gates that ask exactly that, and I will come back to them.

The four laws this protects you from

These are quoted exactly as written, then explained. The verbatim text is on the Garnishments screen as well, so you never have to take my summary on trust.

1. The twenty-day answer duty — RCW 26.18.110(1)

"An employer upon whom service of a wage assignment order or income withholding order has been made shall answer the order by sworn affidavit within twenty days after the date of service."

In plain English: twenty days from the day it is *served* — not the day you opened the envelope — and it must be a **sworn affidavit**. Not a phone call. Not an email to the caseworker.

2. What it costs to miss it — RCW 26.18.110(6)

"An employer who fails to withhold earnings as required by a wage assignment order or income withholding order issued under this chapter may be held liable to the obligee for one hundred percent of the support or maintenance debt... if the employer: ... (b) Fails or refuses to submit an answer to the notice of wage assignment or income withholding after being served."

In plain English, and this is the sentence that justifies the entire feature: read trigger (b) carefully. Failing to *answer* is its own independent route into this liability, completely separate from failing to withhold. **You could withhold every single penny perfectly, on time, every payday — and still be made to pay someone else's entire child support debt because you never filed a piece of paper.** Greenway pays the debt. Plus costs, interest, and their attorney's fees.

That is why this system now nags you daily and never gives up on its own.

3. The creditor version — RCW 6.27.200

"If the garnishee fails to answer the writ within the time prescribed in the writ... it shall be lawful for the court to render judgment by default against such garnishee... for the full amount claimed by the plaintiff against the defendant."

In plain English: in an ordinary creditor garnishment you are the "garnishee". Miss the deadline and the court can enter judgment against Greenway for **the full amount your employee owes** — not the slice you should have taken from one paycheck, the whole debt.

Note the phrase **"the time prescribed in the writ"**. It is printed on the paper and it is **not always twenty days**. This system will therefore never invent that date for you, which I explain below.

4. The deadline nobody expects — RCW 6.27.350(1)

"...such lien shall continue as to subsequent nonexempt earnings until the total subject to the lien equals the amount stated on the writ of garnishment or until the expiration of the employer's payroll period ending on or before sixty days after the effective date of the writ, whichever occurs first."

In plain English: a creditor garnishment is **not permanent**. It dies at sixty days. Keep withholding after that and you are taking money you have no authority to take — a wage claim under RCW 49.52, which reaches you personally, not just the company.

Child support is the **opposite**: it continues until the court or the registry tells you to stop. Same envelope, same drawer, opposite rules. This is the one alert on the list about **stopping** rather than starting, and it is the one almost every employer gets wrong.

How you will actually be notified

You asked where this should live. I did not build a new notification system, because you already have one and it is good. Greenway has run a daily reminder engine for some time. It already emails and push-notifies you about the weekly CCRS upload window and the monthly LIQ-1295 excise return, it already refuses to send the same message twice, and it already has urgency levels.

It knew nothing about payroll. So wage orders became its **fourth** job rather than a second system to maintain. In its own protected block, so that if the payroll read ever fails, your CCRS and excise reminders still go out.

The escalation ladder

One flat alert would be useless. An alarm that sounds the same on day one as on day twenty teaches you to ignore it, and a person who has learned to ignore an alert is in a **worse** position than one who never had an alert at all. So the volume is tied to the consequence:

When	What happens	Why
Days 1 to 10 after service	Silence	Nothing useful to say yet. Saying it anyway is how the later messages get ignored.
10 days before the deadline	One low-key email	Enough time to find the paperwork and post it without rearranging your week. This message should be doing all the work.
5 days before	A warning	Still comfortable, but this stops being a next-week problem. One message per rung — it does not repeat daily at this level.
2 days before, and on the day	Critical, daily , email and push	Remaining time is now measured in postal collections. Interrupting your day is cheaper than what it is protecting against.
Every day after the deadline, indefinitely	Critical, daily, and it never stops on its own	Because the liability does not stop on its own.
3 days after a creditor writ is	One prompt: read the writ,	Because the system will not invent that date.

entered	diary the date	
10 days , then 3 days before a creditor writ expires	Warning, then critical	The writ dies at sixty days and withholding past it is taking wages with no authority.

Every number in that table is pulled directly out of the running code. There is a test that fails if the documentation and the code ever drift apart, because a report telling you that you get ten days of warning when the system gives you seven is worse than no report at all.

Why the overdue alarm never gives up

This was a deliberate decision and you should know I made it. Most software stops nagging after a week or two. This does not, because the RCW 26.18.110(6)(b) exposure is the entire support debt, it **does not expire**, and filing late still helps you. A reminder that quietly gave up after seven days would go silent at exactly the moment the problem was at its worst.

The off switch — and why it is not a "dismiss" button

If an alarm cannot be switched off, you will learn to ignore it within a week, and then it protects nobody. So there has to be an off switch. **Which off switch is the entire design of this slice.**

The obvious one is a **Dismiss** button. It would have been wrong, and wrong in the direction that costs money.

Dismissing records that you saw a message. It does not record that you did the thing. Those are different facts with very different consequences. If a court asks Greenway why no answer was received, "*Michael clicked dismiss on 14 January*" is not a defence. "*The affidavit was filed on 14 January*" is.

So the only way to stop an answer reminder is to record the fact that ends the duty. On each order you will find **"Record the answer"**, which asks for:

- **the date you filed it** — the fact that actually discharges the duty
- **a note** (optional) — how it went, who you sent it to
- or, if genuinely no answer is owed, a **tick-box and a written reason**

Recording it stops the reminders immediately, because the duty is now genuinely discharged and there is a dated record saying so.

There is no snooze anywhere in this feature. That is not an oversight, and I have made it hard to add one by accident: a test reads the source of every file in the answer path on every commit and **fails the build** if the words snooze, dismiss, mute, hide-until or remind-me-later ever appear in code there. A future maintainer looking at a noisy inbox will be tempted, and the temptation will look like kindness.

Five ways the form refuses you, and why each one is a favour

The form checks what you type *before* it saves, and when it refuses it tells you what to do about it. These are worth reading, because each one represents a real way to damage your own defence.

- 1. "You have said both that you filed it and that no answer was required."** Two contradictory claims. One of them is false and the record must not contain both.
- 2. "This is a support order. The answer requirement cannot be waived."** This is the most important refusal in the system. Under RCW 26.18.110(6)(b) a child support or spousal support order **always** carries an answer duty, and no reason of any length can waive it. Ticking "no answer required" on a support order and typing something plausible would silence the one alarm standing between you and liability for someone else's entire support debt. The system will not let you, and it explains why rather than just saying no.
- 3. "The reason is too short."** If you are switching off a legal alarm, the record needs to say *why* in enough words to still make sense to you in eighteen months.
- 4. "You have entered a date before the order was served."** You cannot answer something that had not arrived yet. The message names **both** dates so you know which one to go and check, rather than saying "invalid date" and leaving you guessing.
- 5. "That date is in the future."** This one protects the whole design. Post-dating would switch the alarm off today for

something you plan to do next week — and next week you would never think about it again, because the system already says it is done. It refuses to record an intention as a fact.

A detail worth mentioning

"Is this date in the future?" is measured against the **server's** date in Pacific time, never your laptop's clock. A computer whose clock is a day fast is ordinary, not exotic, and on such a machine the form would happily accept a date the server then rejected — you would watch the software contradict itself with no way to tell which half was lying. There is a test that fails if a clock reading ever appears in either of those screen files.

The one place this system refuses to help you

Creditor writs get a prompt to **read the writ and diary the date** rather than a countdown, and that is deliberate.

RCW 6.27.200 sets the deadline as "*the time prescribed in the writ*". It is often twenty days. It is **not always** twenty days. This system does not have the writ and cannot read it, so producing a confident countdown would mean inventing a legal deadline — and a wrong deadline delivered confidently is worse than no deadline at all, because you would stop looking at the paper.

So three days after you enter a creditor writ, it asks you to read the date off the document while the envelope is still on your desk. It would rather admit what it does not know than guess.

How I know this actually works

A passing test suite proves nothing until you have watched it fail. So I wrote a script that deliberately breaks this feature in **twenty-four** different ways — each one a mistake a real person could plausibly make — and confirmed the tests catch every one.

These are not typos. Every one of them leaves an application that compiles cleanly, passes type-checking, and looks completely finished on screen:

- The cron stops sending wage order emails (the only symptom is **silence**, and silence is also what a compliant month looks like)
- Recording an answer no longer switches the alarm off
- The overdue alarm gives up after thirty days
- A creditor writ is given an invented twenty-day deadline
- A support order becomes waivable
- A post-dated answer is accepted
- The reader stops loading the answer columns
- The button is disconnected from the code that saves — at each of four separate hops
- The alert strip is switched off with `false &&` while still sitting in the file
- The documentation's numbers stop matching the code's

All twenty-four are caught. But the honest and more useful part is what happened on the way there.

Four of them got through the first time

When I first ran the sabotage script, **five** mutations survived. That was the script doing its job — it found real holes in tests that were showing a comforting green tick.

1. **Disconnecting the button from the save code was not caught.** The test counted how many times the function *name* appeared across the files. Cutting the connection left the import line and the comments in place, so the count stayed healthy while the button did nothing. Counting a name proves somebody wrote it down; it does not prove anybody calls it. Replaced with a test that walks all four hops individually, so a break now names the broken link.
2. **Switching off the alert strip with `false &&` was not caught.** Nobody deletes a feature they are unsure about — they comment it out and mean to come back. The old test asked "is this rendered anywhere in the file?" and the answer was still yes; it was just unreachable. Now there is a check for deliberately disabled branches across the whole feature.
3. **Adding a snooze was not caught.** The test looked for the exact word `snooze`, and my sabotage added `snoozeUntil` — which is not the word `snooze`. That is not a contrived spelling; it is the *first* name any developer would type. The check

was looking for the one identifier nobody would ever actually write. It now matches realistic names, and it ignores prose, so that the paragraphs in the source explaining why there is no dismiss button do not trigger the very check that protects them. There is also now a test that proves the detector can still detect — because "found nothing" and "can no longer see anything" look identical from the outside.

4. **One survivor was my own mistake, and I want to be straight about it.** I wrote a sabotage that only changed a type annotation. It changed nothing about what the program does at run time, so no test could possibly have caught it — there was nothing to catch. The right response was to replace the bad sabotage, not to weaken a test chasing a phantom. It now sets the date from UTC instead of Pacific time, which is a genuine bug: UTC runs up to eight hours ahead of Port Orchard, so from about 4pm your board would start stamping tomorrow's date. That version is caught.

Three real holes and one bad experiment. All four fixed, and the fixes are permanent.

What is on your screen now

The board header tells you how many answers are outstanding, in red when it is not zero.

"Deadlines that need you" sits at the top of the left column and only appears when something genuinely needs you. Each item names the employee, the case number, what is required, what happens if it is ignored, and which statute says so.

On each order card, above the End/Pause/Resume buttons, is the answer control. It is placed above them on purpose: a dangerous "end this order" button should not be the nearest thing to your thumb on a row that is asking you for an affidavit.

Down the right-hand side: a four-step checklist for the envelope, the full escalation ladder, three worked examples with real dates, all four statutes quoted in full, and the traps — including the one I would most want you to read, which is that **withholding perfectly is not answering**.

What I decided on your behalf

You gave me executive decision-making power on this slice, so here is a plain list of what I chose and why, in case you disagree with any of it:

1. **Extended your existing reminder engine rather than building a new one.** One system to maintain, one place notifications come from, and your existing CCRS/excise reminders are protected from any payroll failure.
 2. **The overdue alarm never gives up.** The liability does not expire, so neither does the reminder.
 3. **No snooze, no dismiss — only recording a fact.** Explained above. This is the decision I feel strongest about.
 4. **Support orders can never be marked "no answer required".** No reason of any length is accepted.
 5. **No invented deadline for creditor writs.** It asks you to read the paper.
 6. **The answer control sits above the End button**, for the thumb-distance reason.
 7. **Recording an answer does not touch net pay** — it changes no withholding, so nothing about anyone's cheque moves.
-

What this does not do

Being explicit, so there are no surprises:

- **It does not file anything for you.** It prepares and reminds; you sign and send. That boundary is deliberate and unchanged.
 - **It does not know what the writ says.** If the writ prescribes fifteen days, the system cannot know that until you read it off the paper.
 - **It cannot tell you the answer was accepted.** It records the date you say you filed it.
 - **It only watches active and paused orders.** Ended orders carry no live duty.
-

In one sentence

The two deadlines that can cost you somebody else's entire debt were being calculated correctly, shown to you once, and then forgotten — and now they follow you onto the board, into your inbox and onto your phone, getting louder as they approach,

until you record the fact that actually makes them stop.